

모듈

1. 표준 모듈

시작하기 전에

- 모듈(module)

- 코드를 분리하고 공유하는 기능
- 표준 모듈
 - 파이썬에 기본적으로 내장된 모듈
- 외부 모듈
 - 사람들이 만들어 공개한 모듈
- 일반적으로 모듈을 가져오는 import 구문은 코드의 가장 위에 작성

```
import 모듈 이름
```

모듈 사용의 기본 : math 모듈

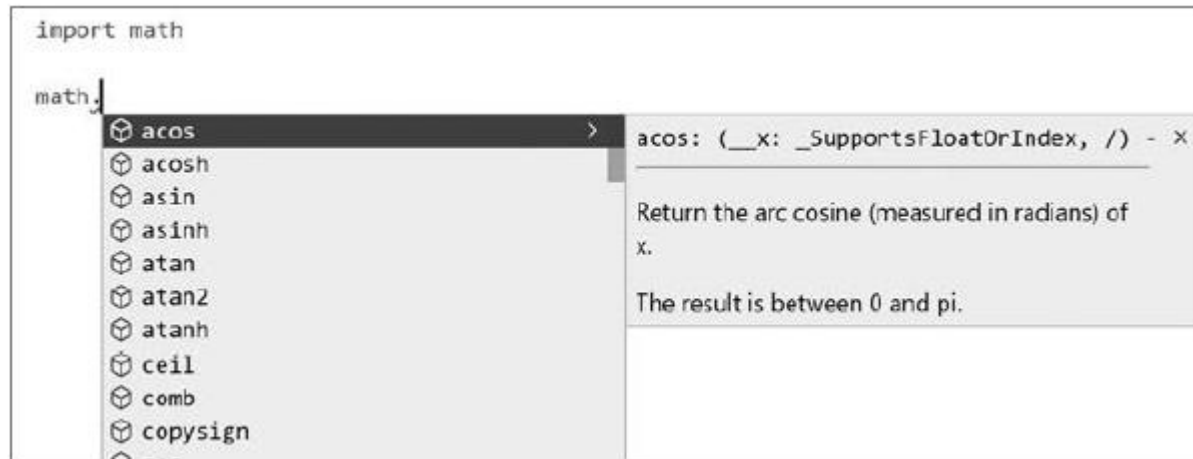
- math 모듈

- 수학과 관련된 기능

```
import math
```

- 여러 변수와 함수를 가진 집합체

- 자동 완성 기능으로 살펴보는 math 모듈의 변수와 함수



모듈 사용의 기본 : math 모듈

- math 모듈을 사용하는 코드

```
>>> import math
```

- 수학/삼각함수

```
>>> math.sin(1)      # 사인
0.8414709848078965
>>> math.cos(1)      # 코사인
0.5403023058681398
>>> math.tan(1)      # 탄젠트
1.5574077246549023
>>>
>>> math.floor(2.5)   # 내림
2
>>> math.ceil(2.5)    # 올림
3
```

모듈 사용의 기본 : math 모듈

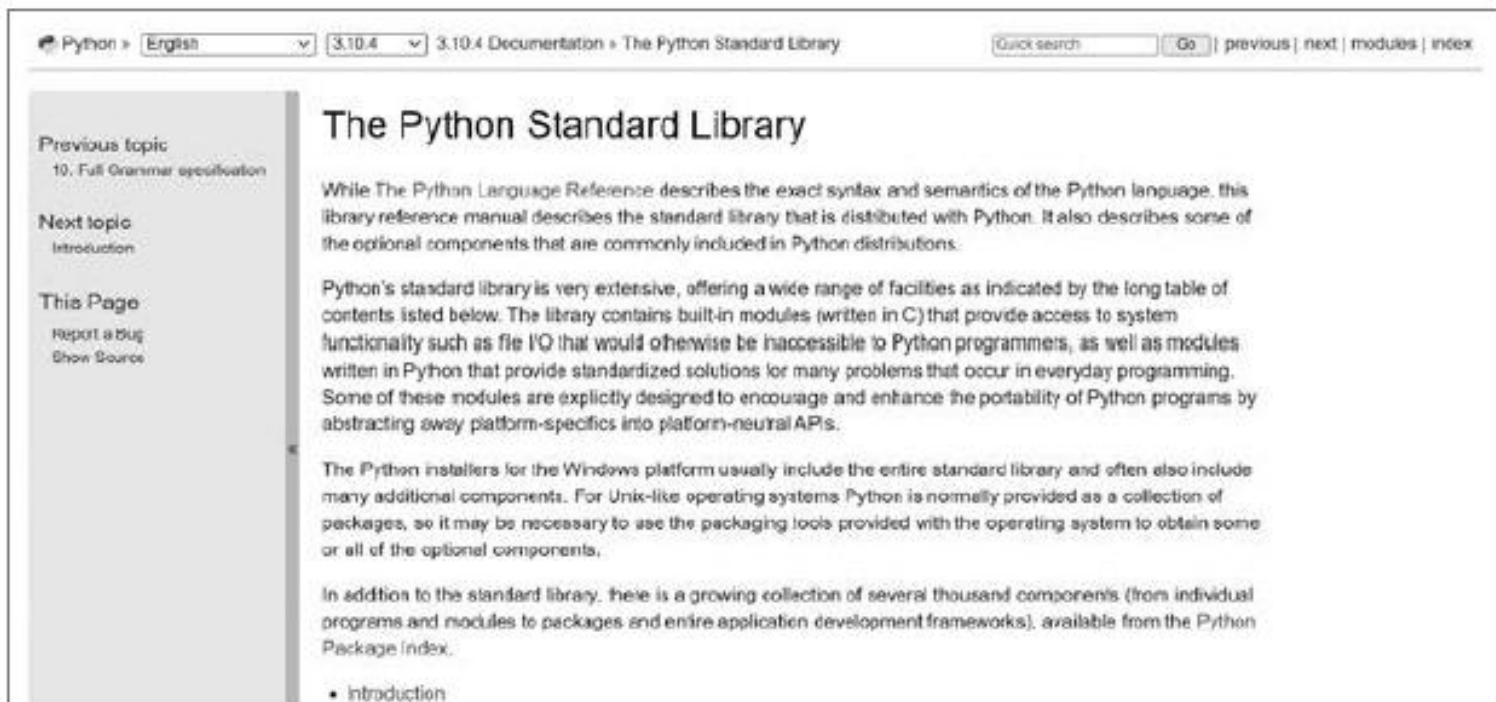
- 모듈 문서

- math 모듈은 다양한 기능 가지고 있음

변수 또는 함수	설명
<code>sin(x)</code>	사인값을 구합니다.
<code>cos(x)</code>	코사인값을 구합니다.
<code>tan(x)</code>	탄젠트값을 구합니다.
<code>log(x[, base])</code>	로그값을 구합니다.
<code>ceil(x)</code>	올림합니다.
<code>floor(x)</code>	내림합니다.

모듈 사용의 기본 : math 모듈

- 파이썬 공식 문서에서 표준 모듈 등 정보 확인 가능
 - <http://docs.python.org/3/library/index.html>



모듈 사용의 기본 : math 모듈

- from 구문

- 다양한 함수를 계속해서 입력하는 것의 비효율성

```
from 모듈 이름 import 가져오고 싶은 변수 또는 함수
```

- '가져오고 싶은 변수 또는 함수'에 여러 개의 변수 또는 함수 입력 가능
- 이를 통해 가져온 기능은 math 붙이지 않고도 사용할 수 있음

```
>>> from math import sin, cos, tan, floor, ceil
>>> sin(1)
0.8414709848078965
>>> cos(1)
0.5403023058681398
>>> tan(1)
1.5574077246549023
>>> floor(2.5)
2
>>> ceil(2.5)
3
```


모듈 사용의 기본 : math 모듈

- as 구문

- 모듈의 이름이 너무 길어 짧게 줄여 사용하고 싶은 경우

```
import 모듈 as 사용하고 싶은 식별자
```

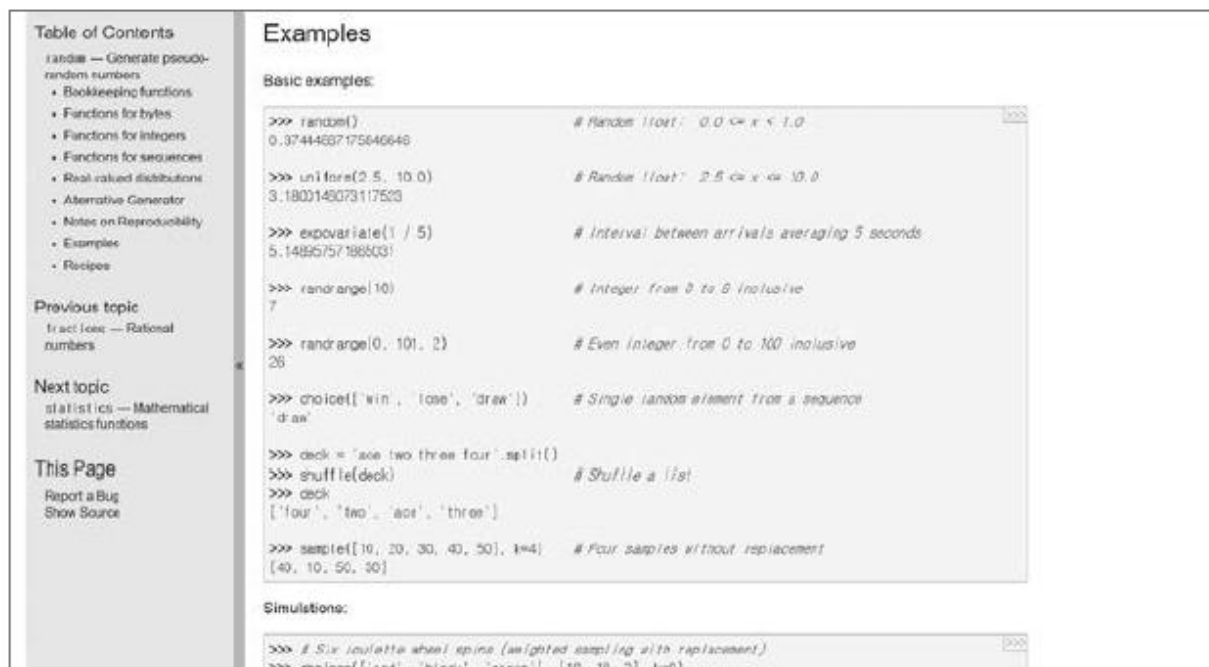
```
>>> import math as m
>>> m.sin(1)
0.8414709848078965
>>> m.cos(1)
0.5403023058681398
>>> m.tan(1)
1.5574077246549023
>>> m.floor(2.5)
2
>>> m.ceil(2.5)
3
```

random 모듈

- random 모듈

```
import random
```

- random 모듈 문서
 - <http://docs.python.org/3/library/random.html#examples-and-recipes>



The screenshot shows the Python documentation for the random module. On the left is a sidebar with a 'Table of Contents' listing various functions like random(), randint(), and shuffle(). Below this are links for 'Previous topic' (fractions) and 'Next topic' (statistics). The main content area is titled 'Examples' and contains two sections: 'Basic examples' and 'Simulations'. The 'Basic examples' section shows several code snippets with their outputs and comments, such as random() returning a float between 0.0 and 1.0, and randrange() returning a random integer. The 'Simulations' section shows a snippet for simulating a roulette wheel spin.

Table of Contents

- random — Generate pseudo-random numbers
 - Bookkeeping functions
 - Functions for bytes
 - Functions for integers
 - Functions for sequences
 - Real-valued distributions
 - Alternative Generator
 - Notes on Reproducibility
 - Examples
 - Recipes

Previous topic
fractions — Rational numbers

Next topic
statistics — Mathematical statistics functions

This Page
Report a Bug
Show Source

Examples

Basic examples:

```
>>> random()                                # Random float: 0.0 <= x < 1.0
0.37444687175646646

>>> uniform(2.5, 10.0)                       # Random float: 2.5 <= x <= 10.0
3.180145073117523

>>> expovariate(1 / 5)                       # Interval between arrivals averaging 5 seconds
5.14957571985031

>>> randrange(10)                             # Integer from 0 to 9 inclusive
7

>>> randrange(0, 101, 2)                      # Even integer from 0 to 100 inclusive
26

>>> choice(['win', 'lose', 'draw'])           # Single random element from a sequence
'draw'

>>> deck = 'ace two three four'.split()
>>> shuffle(deck)                             # Shuffle a list
>>> deck
['four', 'two', 'ace', 'three']

>>> sample([10, 20, 30, 40, 50], k=4)          # Four samples without replacement
[40, 10, 50, 30]
```

Simulations:

```
>>> # Six roulette wheel spins (weighted sampling with replacement)
>>> choices(['red', 'black', 'green'], [18, 18, 2], k=6)
```

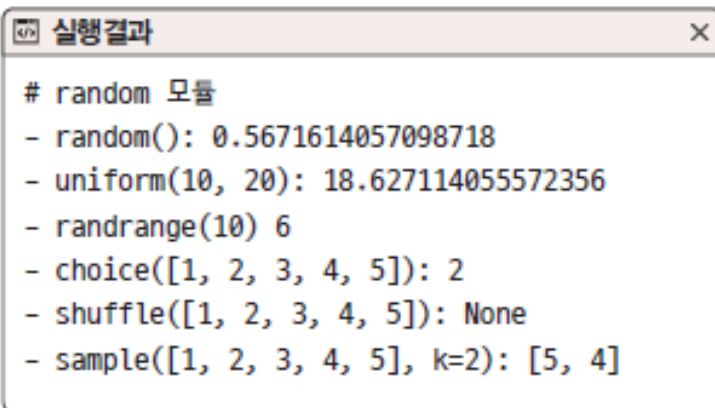
random 모듈

- 예시 - random 모듈

```
01  import random
02  print("# random 모듈")
03
04  # random(): 0.0 <= x < 1.0 사이의 float를 리턴합니다.
05  print("- random():", random.random())
06
07  # uniform(min, max): 지정한 범위 사이의 float를 리턴합니다.
08  print("- uniform(10, 20):", random.uniform(10, 20))
09
10  # randrange(): 지정한 범위의 int를 리턴합니다.
11  # - randrange(max): 0부터 max 사이의 값을 리턴합니다.
12  # - randrange(min, max): min부터 max 사이의 값을 리턴합니다.
13  print("- randrange(10)", random.randrange(10))
```

random 모듈

```
14
15 # choice(list): 리스트 내부에 있는 요소를 랜덤하게 선택합니다.
16 print("- choice([1, 2, 3, 4, 5]):", random.choice([1, 2, 3, 4, 5]))
17
18 # shuffle(list): 리스트의 요소들을 랜덤하게 섞습니다.
19 print("- shuffle([1, 2, 3, 4, 5]):", random.shuffle([1, 2, 3, 4, 5]))
20
21 # sample(list, k=<숫자>): 리스트의 요소 중에 k개를 뽑습니다.
22 print("- sample([1, 2, 3, 4, 5], k=2):", random.sample([1, 2, 3, 4, 5], k=2))
```



실행결과

```
# random 모듈
- random(): 0.5671614057098718
- uniform(10, 20): 18.627114055572356
- randrange(10) 6
- choice([1, 2, 3, 4, 5]): 2
- shuffle([1, 2, 3, 4, 5]): None
- sample([1, 2, 3, 4, 5], k=2): [5, 4]
```

random 모듈

- 앞 코드 5행의 random.random()처럼 random을 계속 입력하는 것은 효율적이지 못하므로 from 구문 활용해서 임포트

```
from time import random, randrange, choice
```

sys 모듈

- sys 모듈

- 시스템과 관련된 정보 가진 모듈
- 명령 매개변수 받을 때 많이 사용

```
01  # 모듈을 읽어 들입니다.  
02  import sys  
03  
04  # 명령 매개변수를 출력합니다.  
05  print(sys.argv)  
06  print("---")  
07  
08  # 컴퓨터 환경과 관련된 정보를 출력합니다.  
09  print("getwindowsversion()", sys.getwindowsversion())  
10  print("---")
```

sys 모듈

```
11 print("copyright:", sys.copyright)
12 print("----")
13 print("version:", sys.version)
14
15 # 프로그램을 강제로 종료합니다.
16 sys.exit()
```

- 5행 sys.argv – 명령 매개변수
 - 아래와 같이 실행하면 ['module_sys.py', '10', '20', '30'] 리스트 들어옴
 - 명령 프롬프트 창의 경로와 실행할 .py 파일이 위치한 경로는 반드시 일치

```
> python module_sys.py 10 20 30
```

sys 모듈

`['module_sys.py', '10', '20', '30']` → 명령 매개변수입니다. 입력한 명령어에 따라 달라집니다.

```
getwindowsversion:() sys.getwindowsversion(major=10, minor=0, build=14393,
platform=2, service_pack='')

```

```
copyright: Copyright (c) 2001-2022 Python Software Foundation.
All Rights Reserved.

```

...생략...

```
version: 3.10.3 (tags/v3.10.3:a342a49, Mar 16 2022, 13:07:40) [MSC v.1929 64
bit (AMD64)]

```


os 모듈

- os 모듈

- 운영체제와 관련된 기능 가진 모듈
- 새로운 폴더 만들거나 폴더 내부 파일 목록 보는 등

```
01  # 모듈을 읽어 들입니다.
02  import os
03
04  # 기본 정보를 몇 개 출력해봅시다.
05  print("현재 운영체제:", os.name)
06  print("현재 폴더:", os.getcwd())
07  print("현재 폴더 내부의 요소:", os.listdir())
08
09  # 폴더를 만들고 제거합니다[폴더가 비어있을 때만 제거 가능].
10  os.mkdir("hello")
11  os.rmdir("hello")
12
```

os 모듈

```
13 # 파일을 생성하고 + 파일 이름을 변경합니다.
14 with open("original.txt", "w") as file:
15     file.write("hello")
16 os.rename("original.txt", "new.txt")
17
18 # 파일을 제거합니다.
19 os.remove("new.txt")
20 # os.unlink("new.txt")
21
22 # 시스템 명령어 실행
23 os.system("dir")
```

현재 운영체제: nt

현재 폴더: C:\Users\hasat\sample

현재 폴더 내부의 요소: ['.vscode', 'beaut.py', 'download-png1.py', 'file.txt', 'freq.json', 'ghostdriver.log', 'iris.csv', 'lang-plot.png', 'mnist', 'mtest.py', 'newFile.xlsx', 'output.png', 'proj', 'rint.py', 'stats_104102.xlsx', 'test', 'test.csv', 'test.html', 'test.png', 'test.py', 'test.rb', 'test.txt', 'test_a.txt', 'train', 'underscore.js', 'Website.png', 'Website_B.png', 'Website_C.png', 'Website_D.png', '__pycache__']

C 드라이브의 볼륨: BOOTCAMP

볼륨 일련 번호: FCCF-6067

C:\Users\hasat\sample 디렉터리

2019-05-01	오전 12:18	<DIR>	.
2019-05-01	오전 12:18	<DIR>	..
...생략...			
2019-05-28	오전 04:49	<DIR>	__pycache__
	24개 파일		1,908,017 바이트
	8개 디렉터리		16,895,188,992 바이트 남음

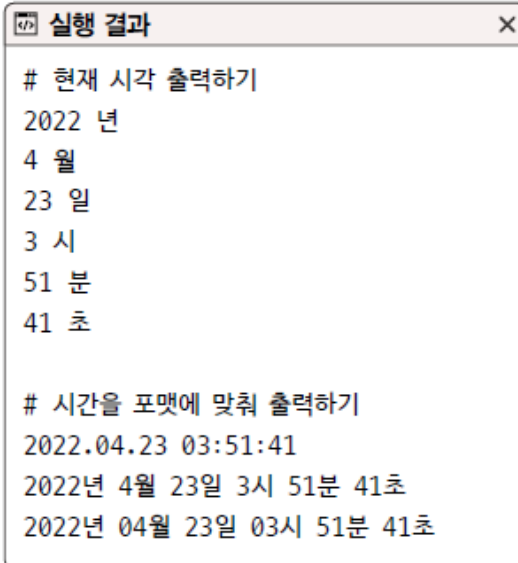
→ 명령 프롬프트에서
그냥 dir을 입력했을 때의
결과와 동일합니다.
단지 파이썬에서
dir 명령어를
호출했을 뿐입니다.

datetime 모듈

- datetime 모듈

- date(날짜) 및 time(시간)과 관련된 모듈로, 날짜 형식 만들 때 자주 사용되는 코드들로 구성

```
01  # 모듈을 읽어 들입니다.  
02  import datetime  
03  
04  # 현재 시각을 구하고 출력하기  
05  print("# 현재 시각 출력하기")  
06  now = datetime.datetime.now()  
07  print(now.year, "년")  
08  print(now.month, "월")  
09  print(now.day, "일")  
10  print(now.hour, "시")  
11  print(now.minute, "분")  
12  print(now.second, "초")  
13  print()
```



```
# 현재 시각 출력하기  
2022 년  
4 월  
23 일  
3 시  
51 분  
41 초  
  
# 시간을 포맷에 맞춰 출력하기  
2022.04.23 03:51:41  
2022년 4월 23일 3시 51분 41초  
2022년 04월 23일 03시 51분 41초
```

datetime 모듈

```
14
15 # 시간 출력 방법
16 print("# 시간을 포맷에 맞춰 출력하기")
17 output_a = now.strftime("%Y.%m.%d %H:%M:%S")
18 output_b = "{}년 {}월 {}일 {}시 {}분 {}초".format(now.year,\
19     now.month,\
20     now.day,\
21     now.hour,\
22     now.minute,\
23     now.second)
24 output_c = now.strftime("%Y{} %m{} %d{} %H{} %M{} %S{}").format(*"년월일시분초")
25 print(output_a)
26 print(output_b)
27 print(output_c)
28 print()
```

↓
문자열, 리스트 등 앞에 *을 붙이면
요소 하나하나가 매개변수로 지정됩니다.

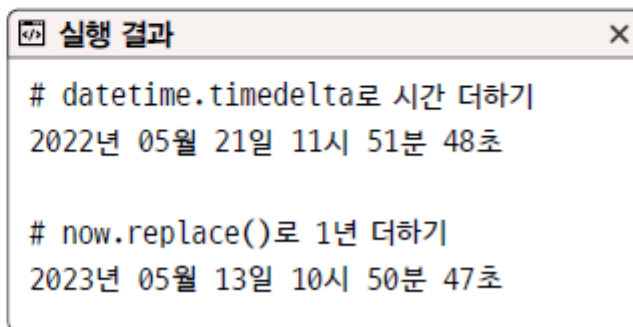
datetime 모듈

- output_a처럼 strftime() 함수 사용하면 시간을 형식에 맞춰 출력 가능
 - 한국어 등의 문자는 매개변수에 넣을 수 없음
- 그 외 다양한 시간 처리 기능

```
01  # 모듈을 읽어 들입니다.
02  import datetime
03  now = datetime.datetime.now()
04
05  # 특정 시간 이후의 시간 구하기
06  print("# datetime.timedelta로 시간 더하기")
07  after = now + datetime.timedelta(\
08      weeks=1,\
09      days=1,\
10      hours=1,\
11      minutes=1,\
12      seconds=1)
```

datetime 모듈

```
13 print(after.strftime("%Y{} %m{} %d{} %H{} %M{} %S{}").format(*"년월일시분초"))
14 print()
15
16 # 특정 시간 요소 교체하기
17 print("# now.replace()로 1년 더하기")
18 output = now.replace(year=(now.year + 1))
19 print(output.strftime("%Y{} %m{} %d{} %H{} %M{} %S{}").format(*"년월일시분초"))
```



실행 결과

```
# datetime.timedelta로 시간 더하기
2022년 05월 21일 11시 51분 48초

# now.replace()로 1년 더하기
2023년 05월 13일 10시 50분 47초
```

- timedelta() 함수 사용하면 특정한 시간의 이전 또는 이후 구함
 - 다만 1년 후, 2년 후 등의 몇 년 후를 구하는 기능은 없음
 - "1년 후" 구할 때는 replace() 함수 사용해 날짜 값을 교체

time 모듈

- time 모듈

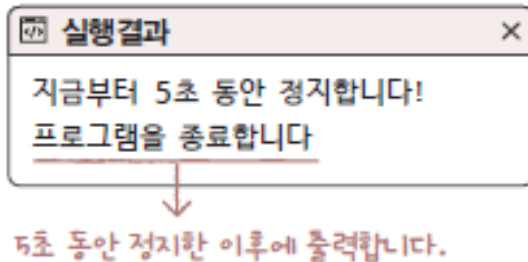
- 시간과 관련된 기능

```
import time
```

- time.sleep() 함수

- 특정 시간 동안 코드 진행을 정지
- 정지하고 싶은 시간을 초 단위로 입력

```
01 import time
02
03 print("지금부터 5초 동안 정지합니다!")
04 time.sleep(5)
05 print("프로그램을 종료합니다")
```



urllib 모듈

- urllib 모듈

- URL 다루는 라이브러리

```
01  # 모듈을 읽어 들입니다.  
02  from urllib import request  
03  
04  # urlopen() 함수로 구글의 메인 페이지를 읽습니다.  
05  target = request.urlopen("https://google.com")  
06  output = target.read()  
07  
08  # 출력합니다.  
09  print(output)
```

urllib 모듈

- urlopen() 함수 : URL 주소의 페이지 열기
 - read() 함수를 호출하면 해당 웹 페이지에 있는 내용을 읽어옴

```
b'<!doctype html><html itemscope="" itemtype="http://schema.org/WebPage"
lang="ko"><head><meta content="text/html; charset=UTF-8" http-equiv="Content-
Type"><meta content="/logos/doodles/2019/amy-johnsons-114th-birthday-
5154304993263616.2-law.gif" itemprop="image">
...생략...
```

바이너리 데이터(binary data)

2. 외부 모듈

모듈 설치하기

- 외부 모듈 설치

- [Windows] + [R] 클릭하여 프로그램 실행창 띄우고 [cmd] 입력하면 나타나는 명령 프롬프트 창에서 실행

`pip install 모듈 이름` python2가 있는 상태에서 python3를 설치했다면 pip3 명령어를 사용하세요.

```
> pip install beautifulsoup4
Collecting beautifulsoup4
  Downloading beautifulsoup4-4.11.1-py3-none-any.whl (128 kB)
...생략...
Installing collected packages: soupsieve, beautifulsoup4
Successfully installed beautifulsoup4-4.11.1
soupsieve-2.3.2.post1
```

모듈 설치하기

- pip
 - 특정 버전 모듈 설치 및 제거 등
 - <https://packaging.python.org/en/latest>



모듈 찾아보기

- 파이썬 관련 도서 구매할 경우 추천 받는 모듈
- 인터넷 커뮤니티 정보로 접하는 모듈
- 직접 구글에서 검색하여 찾는 모듈
 - (Python + 키워드) 검색

라이브러리와 프레임워크

- 라이브러리(library)

- 정상적인 제어 하는 모듈
 - 개발자가 모듈의 기능을 호출하는 형태

- 프레임워크(framework)

- 제어 역전이 발생하는 모듈
 - 모듈이 개발자가 작성한 코드를 실행하는 형태

- 제어 역전(LoC : Invasion of Control)

- 개발자가 모듈의 함수를 호출하는 것이 일반적인 제어 흐름.
- 반대로 개발자가 만든 함수를 모듈이 실행하는 것은 제어가 역전된 것

라이브러리와 프레임워크

- 라이브러리

- 개발자가 모듈 기능을 호출하는 형태의 모듈

정상적인 제어 = 라이브러리

```
# 모듈을 읽어 들입니다.  
from math import sin, cos, tan, floor, ceil  
  
# sin, cos, tan를 구합니다.  
print("sin(1):", sin(1))  
print("cos(1):", cos(1))  
print("tan(1):", tan(1))  
  
# 내림과 올림을 구합니다.  
print("floor(2.5):", floor(2.5))  
print("ceil(2.5):", ceil(2.5))
```

라이브러리와 프레임워크

- 프레임워크

```
from flask import Flask
app = Flask(__name__)

@app.route("/")
def hello():
    return "<h1>Hello World!</h1>"
```

- 내부에 함수만 정의하고 직접적으로 무언가 실행하는 코드 없음

라이브러리와 프레임워크

```
> set FLASK_APP=flask_basic.py
> flask run
* Serving Flask app "flask_basic.py"
* Running on http://127.0.0.1:5000/ (Press CTRL+C to quit)
```

- Flask 모듈 내부에서 출력된 글자들
- Flask가 내부적으로 서버를 실행한 뒤 지정한 파일을 읽어 들여 적절한 상황에 스스로 실행
- 프레임워크 : 모듈이 개발자가 작성한 코드를 실행하는 형태의 모듈
- 제어 역전 : 개발자가 만든 함수를 모듈이 실행하는 형태의, 일반적인 경우와 반대인 제어 흐름

함수 데코레이터

- 데코레이터 : @로 시작하는 구문
- 함수 데코레이터의 기본
 - "hello"를 출력하는 함수

```
def hello():  
    print("hello")
```

함수 데코레이터

- 예시 - 함수 데코레이터의 기본

```
01  # 함수 데코레이터를 생성합니다.
02  def test(function):
03      def wrapper():
04          print("인사가 시작되었습니다.")
05          function()
06          print("인사가 종료되었습니다.")
07      return wrapper
08
09  # 데코레이터를 붙여 함수를 만듭니다.
10  @test
11  def hello():
12      print("hello")
13
14  # 함수를 호출합니다.
15  hello()
```

실행 결과

인사가 시작되었습니다.
hello
인사가 종료되었습니다.

함수 데코레이터

● 데코레이터 장점

- functools라는 모듈을 사용할 수 있음
- 매개변수 등을 전달할 수 있어 반복되는 구문이 많아질 때 소스의 가독성을 높임

```
# 모듈을 가져옵니다.  
from functools import wraps  
  
# 함수로 데코레이터를 생성합니다.  
def test(function):  
    @wraps(function)  
    def wrapper(*arg, **kwargs):  
        print("인사가 시작되었습니다.")  
        function(*arg, **kwargs)  
        print("인사가 종료되었습니다.")  
    return wrapper
```

→ 가변 매개변수와 키워드 가변 매개변수를 사용하는 코드입니다. 이와 관련된 내용은 고급 주제이므로 이 책에서는 다루지 않습니다.

3. 모듈 만들기

시작하기 전에

- module_basic 디렉터리 만든 후 아래 두 파일 넣기
 - module_basic / test_module.py



main.py



test_module.py

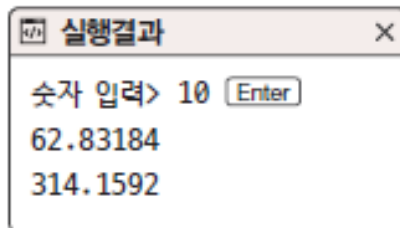
모듈 만들기

- 예시 - 쉬운 모듈 만들기

```
01  # test_module.py 파일
02  PI = 3.141592
03
04  def number_input():
05      output = input("숫자 입력> ")
06      return float(output)
07
08  def get_circumference(radius):
09      return 2 * PI * radius
10
11  def get_circle_area(radius):
12      return PI * radius * radius
```

모듈 만들기

```
01  # main.py 파일
02  import test_module as test
03
04  radius = test.number_input()
05  print(test.get_circumference(radius))
06  print(test.get_circle_area(radius))
```



- 패키지(package)
 - 복잡하고 구조화된 모듈 만들 때 사용하는 기능

__name__ == "__main__"

- __name__

- 엔트리 포인트(entry point) / 메인(main)
 - 프로그램의 진입점
 - 메인 내부에서의 __name__은 "__main__"

```
>>> __name__  
'__main__'
```

- 모듈의 __name__

- 엔트리 포인트 아니지만 엔트리 포인트 파일 내에서 import 되었기 때문에 모듈 내 코드가 실행
- 모듈 내부에서 __name__ 출력하면 모듈의 이름 나타냄

__name__ == "__main__"

- 예시 – 모듈 이름을 출력하는 모듈 만들기

```
01  # main.py 파일
02  import test_module
03
04  print("# 메인의 __name__ 출력하기")
05  print(__name__)
06  print()
```

```
01  # test_module.py 파일
02  print("# 모듈의 __name__ 출력하기")
03  print(__name__)
04  print()
```

__name__ == "__main__"

A screenshot of a Python Shell window titled "실행결과" (Execution Result). The window contains two lines of output. The first line is a comment "# 모듈의 __name__ 출력하기" (Print module's __name__) followed by the output "test_module". The second line is a comment "# 메인의 __name__ 출력하기" (Print main's __name__) followed by the output "__main__".

```
# 모듈의 __name__ 출력하기
test_module

# 메인의 __name__ 출력하기
__main__
```

__name__ == "__main__"

- __name__ 활용하기

- 엔트리 포인트 파일 내부에서 __name__이 "__main__" 값을 가짐을 활용하여 현재 파일이 모듈로 실행되는지 엔트리 포인트로 실행되는지 확인
- 예시 – 모듈 활용하기 소스 코드 `module_example/test_module.py`

```
01  PI = 3.141592
02
03  def number_input():
04      output = input("숫자 입력> ")
05      return float(output)
06
07  def get_circumference(radius):
08      return 2 * PI * radius
09
10  def get_circle_area(radius):
```

`module_example` 디렉토리를 생성해 파일을 저장

`__name__ == "__main__"`

```
11     return PI * radius * radius
```

```
12
```

```
13 # 활용 예
```

```
14 print("get_circumference(10):", get_circumference(10))
```

```
15 print("get_circle_area(10): ", get_circle_area(10))
```

"이런 식으로 동작해요!"를 알려주는
활용 예를 넣었습니다.



```
01 import test_module as test → 위 모듈을 읽어 들입니다.
```

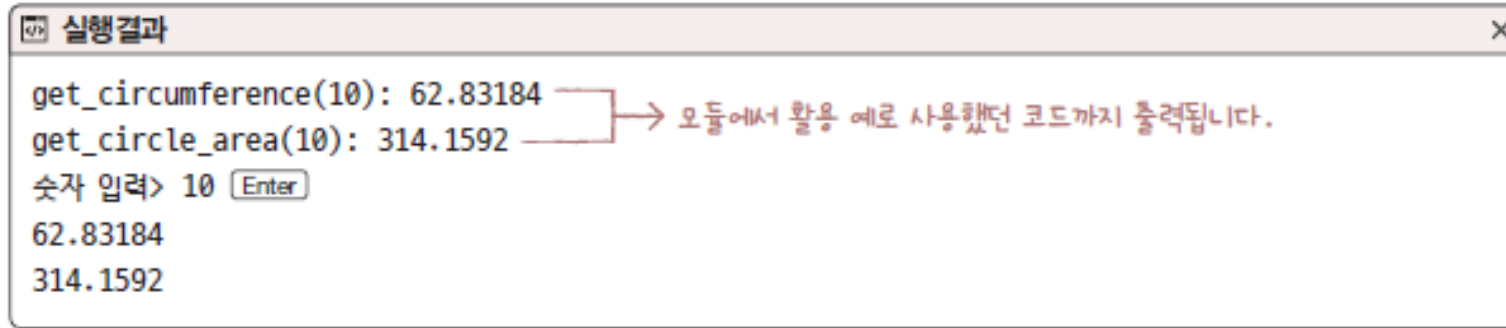
```
02
```

```
03 radius = test.number_input()
```

```
04 print(test.get_circumference(radius))
```

```
05 print(test.get_circle_area(radius))
```

__name__ == "__main__"



```
실행결과
get_circumference(10): 62.83184
get_circle_area(10): 314.1592
숫자 입력> 10 Enter
62.83184
314.1592
```

- 현재 test_module.py 파일에는 동작 설명을 위해 추가한 활용 예시 부분 존재
- 모듈로 사용하고 있는데 내부에서 출력 발생하여 문제
- 현재 파일이 엔트리 포인트인지 구분하는 코드 활용
- 조건문으로 __name__이 "__main__"인지 확인

__name__ == "__main__"

- 예시 – 엔트리 포인트를 확인하는 모듈 만들기

```
01  PI = 3.141592
02
03  def number_input():
04      output = input("숫자 입력> ")
05      return float(output)
06
07  def get_circumference(radius):
08      return 2 * PI * radius
09
10  def get_circle_area(radius):
11      return PI * radius * radius
```

```
12  # 활용 예
```

```
13  if __name__ == "__main__":
14      print("get_circumference(10):", get_circumference(10))
15      print("get_circle_area(10): ", get_circle_area(10))
```

현재 파일이 엔트리 포인트인지 확인하고,
엔트리 포인트일 때만 실행합니다.



__name__ == "__main__"

```
01  import test_module as test
02
03  radius = test.number_input()
04  print(test.get_circumference(radius))
05  print(test.get_circle_area(radius))
```

실행결과

숫자 입력> 10

62.83184

314.1592

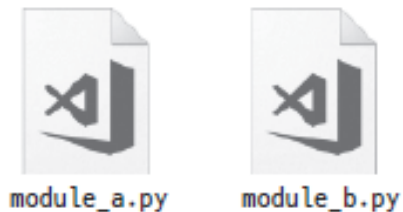
패키지

- 모듈(module)
- 패키지 관리 시스템(Package Management System)
 - pip(Package Installer for Python)
 - 모듈이 모여서 구조 이루면 패키지
- 패키지 만들기
 - main.py 파일은 엔트리 포인트로, test_package 폴더는 패키지로 사용



패키지

- test_package 폴더 내부에 module_a.py 파일과 module_b.py 파일 생성



- module_a.py 파일에 입력

```
01  # ./test_package/module_a.py의 내용
02  variable_a = "a 모듈의 변수"
```

- module_b.py 파일에 입력

```
01  # ./test_package/module_b.py의 내용
02  variable_b = "b 모듈의 변수"
```

패키지

- main.py 파일에 입력

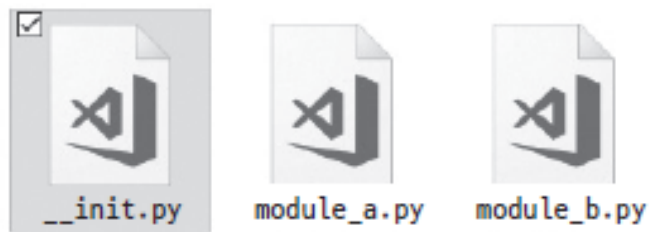
```
01  # 패키지 내부의 모듈을 읽어 들입니다.  
02  import test_package.module_a as a  
03  import test_package.module_b as b  
04  
05  # 모듈 내부의 변수를 출력합니다.  
06  print(a.variable_a)  
07  print(b.variable_b)
```



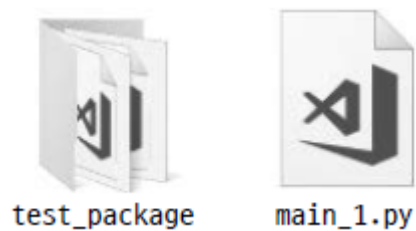
패키지

- `__init__.py` 파일

- 패키지 읽을 때 어떤 처리를 수행해야 하거나 패키지 내부의 모듈들을 한꺼번에 가져오고 싶을 때 사용
- `test_package` 폴더 내부에 `__init__.py` 파일 추가



- `test_package` 폴더 외부에 엔트리 포인트로 사용할 `main_1.py` 생성



패키지

- `__init__.py`

```
01  # "from test_package import *"로
02  # 모듈을 읽어 들일 때 가져올 모듈
03  __all__ = ["module_a", "module_b"] → *사용 시 읽어들일 모듈의 목록
04
05  # 패키지를 읽어 들일 때 처리를 작성할 수도 있습니다.
06  print("test_package를 읽어 들였습니다.")
```

- `main_1.py`

```
01  # 패키지 내부의 모듈을 모두 읽어 들입니다.
02  from test_package import *
03
04  # 모듈 내부의 변수를 출력합니다.
05  print(module_a.variable_a)
06  print(module_b.variable_b)
```

실행결과

```
test_package를 읽어 들였습니다.
a 모듈의 변수
b 모듈의 변수
```

텍스트 데이터

- 텍스트 데이터(text data)

- 우리가 쉽게 읽을 수 있는 형태의 데이터
 - 컴퓨터는 내부적으로 모든 처리를 0과 1로 이루어진 이진숫자로 수행
- 텍스트 데이터로 쉽게 편집

“Hello Python Programming”의 이진 데이터

```
01001000 01100101 01101100 01101100 01101111 00100000 01010000 01111001 01110100
01101000 01101111 01101110 00100000 01010000 01110010 01101111 01100111 01110010
01100001 01101101 01101101 01101001 01101110 01100111
```

“Hello Python Programming”의 이진 데이터를 10진수로 표기한 형태

```
72 101 108 108 111 32 80 121 116 104 111 110 32 80 114 111 103 114 97 109 109 105
110 103
```

텍스트 데이터

Ctrl	Dec	Hex	Char	Code	Dec	Hex	Char	Dec	Hex	Char	Dec	Hex	Char
^@	0	00		NUL	32	20	!	64	40	@	96	60	'
^A	1	01		SOH	33	21	!	65	41	A	97	61	a
^B	2	02		STX	34	22	"	66	42	B	98	62	b
^C	3	03		ETX	35	23	#	67	43	C	99	63	c
^D	4	04		EOT	36	24	\$	68	44	D	100	64	d
^E	5	05		ENQ	37	25	%	69	45	E	101	65	e
^F	6	06		ACK	38	26	&	70	46	F	102	66	f
^G	7	07		BEL	39	27	.	71	47	G	103	67	g
^H	8	08		BS	40	28	(	72	48	H	104	68	h
^I	9	09		HT	41	29	)	73	49	I	105	69	i
^J	10	0A		LF	42	2A	*	74	4A	J	106	6A	j
^K	11	0B		VT	43	2B	+	75	4B	K	107	6B	k
^L	12	0C		FF	44	2C	,	76	4C	L	108	6C	l
^M	13	0D		CR	45	2D	-	77	4D	M	109	6D	m
^N	14	0E		SO	46	2E	.	78	4E	N	110	6E	n
^O	15	0F		SI	47	2F	/	79	4F	O	111	6F	o
^P	16	10		DLE	48	30	0	80	50	P	112	70	p
^Q	17	11		DC1	49	31	1	81	51	Q	113	71	q
^R	18	12		DC2	50	32	2	82	52	R	114	72	r
^S	19	13		DC3	51	33	3	83	53	S	115	73	s
^T	20	14		DC4	52	34	4	84	54	T	116	74	t
^U	21	15		NAK	53	35	5	85	55	U	117	75	u
^V	22	16		SYN	54	36	6	86	56	V	118	76	v
^W	23	17		ETB	55	37	7	87	57	W	119	77	w
^X	24	18		CAN	56	38	8	88	58	X	120	78	x
^Y	25	19		EM	57	39	9	89	59	Y	121	79	y
^Z	26	1A		SUB	58	3A	:	90	5A	Z	122	7A	z
^[	27	1B		ESC	59	3B	;	91	5B	[	123	7B	{
^\	28	1C		FS	60	3C	<	92	5C	\	124	7C	
^]	29	1D		GS	61	3D	=	93	5D	]	125	7D	}
^^	30	1E	▲	RS	62	3E	>	94	5E	^	126	7E	~
^-	31	1F	▼	US	63	3F	?	95	5F	_	127	7F	

MSDN ASCII Character Codes Chart 1

바이너리 데이터

- 바이너리 데이터(binary data)

- 텍스트 에디터로 열었을 때 의미를 이해할 수 없는 데이터
- 이미지, 동영상 등

3바이트를 차지하는 텍스트 데이터 100[49 48 48]

00110001 00110000 00110000

1바이트를 차지하는 바이너리 데이터 100[100]

01100100

바이너리 데이터

비교 항목	텍스트 데이터	바이너리 데이터
구분 방법	• 텍스트 에디터로 열었을 때 읽을 수 있습니다.	• 텍스트 에디터로 열어도 읽을 수 없습니다.
장점	• 사람이 쉽게 읽을 수 있습니다. • 텍스트 에디터로 쉽게 편집할 수 있습니다.	• 용량이 적습니다.
단점	• 용량이 큼니다.	• 사람이 쉽게 읽을 수 없습니다. • 일반적으로는 텍스트 에디터로 편집할 수 없습니다.

● 인코딩과 디코딩

- 인코딩(encoding)
 - 텍스트 데이터를 우리가 읽기 쉬운 글자로 보이기 위해, 혹은 바이너리 데이터를 읽어 이미지로 보이기 위해 데이터를 변환하는 것
- 디코딩(decoding)
 - 반대 과정

바이너리 데이터

- 텍스트 데이터와 바이너리 데이터
- 인터넷의 이미지 저장하기
 - 파일 열 때 뒤에 "b" 붙이기
 - 바이너리 데이터로 저장

모듈을 분석하는 방법

- pip list 명령어 사용하여 설치된 명령어 확인

```
> pip list
astroid (1.5.2)
beautifulsoup4 (4.6.0)
certifi (2017.4.17)
chardet (3.0.4)
...생략...
```

모듈을 분석하는 방법

- pip show <설치된 모듈> 입력하여 모듈 설치된 위치 확인

```
> pip show beautifulsoup4
Name: beautifulsoup4
Version: 4.6.0
Summary: Screen-scraping library
Home-page: http://www.crummy.com/software/BeautifulSoup/bs4/
...생략...
Location: c:\users\hasat\appdata\local\programs\python\python36-32\lib\site-
packages

Requires:
```